

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ «Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Прикладная информатика

О Т Ч Е Т

**Отчёт по курсовой работе дисциплины Мобильная
разработка (Android и IOs) на тему “Разработка Android-
приложения Мессенджер”**

Обучающийся:

Русинов Василий Александрович, К3440

Преподаватель:

Шатинский Григорий Сергеевич

Санкт-Петербург
2026

АННОТАЦИЯ

В рамках курсовой работы разработано Android-приложение MessengerApp, имитирующее базовый функционал мессенджера: отображение ленты сообщений, редактирование профиля пользователя и настройка параметров приложения. Проект реализован поэтапно в формате лабораторных работ с постепенным усложнением: создание интерфейса и навигации, внедрение архитектуры MVVM, интеграция с REST API и локальным хранилищем Room, а также реализация фоновой синхронизации через WorkManager и системы уведомлений.

В ходе разработки использованы Kotlin, Android Jetpack (ViewModel, LiveData, Navigation Component, Room, WorkManager), Retrofit и OkHttp для сетевого взаимодействия, а также компоненты Material Design 3 для построения современного UI. Результатом является масштабируемая архитектура с разделением ответственности между слоями и поддержкой офлайн-режима.

СОДЕРЖАНИЕ

1 ОБЗОР ТЕХНОЛОГИЙ И ИНСТРУМЕНТОВ	6
1.1 Платформа Android и среда разработки	6
1.2 Архитектурный паттерн MVVM	6
1.3 Android Jetpack	7
1.4 Retrofit и OkHttp	7
1.5 Material Design 3	7
1.6 Работа с изображениями	7
2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ.....	9
2.1 Общая структура и пакеты.....	9
2.2 Presentation Layer.....	9
2.3 Domain Layer	9
2.4 Data Layer	10
2.5 Background Layer	10
2.6 Диаграмма потока данных	10
3 РЕАЛИЗАЦИЯ ЛАБОРАТОРНЫХ РАБОТ	11
3.1 Лабораторная работа №1: базовый UI и Material Design.....	11
3.2 Лабораторная работа №2: интеграция с API и Room.....	12
3.3 Лабораторная работа №3: закрепление API и Room.....	13
3.4 Лабораторная работа №4: WorkManager и уведомления.....	13
4 ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС	16
4.1 Главный экран - лента сообщений	16
4.2 Экран профиля	17

4.3 Экран настроек	18
4.4 Bottom Navigation	19
4.5 Уведомления	19
5 ОСОБЕННОСТИ РЕАЛИЗАЦИИ	21
5.1 Кеширование и Single Source of Truth	21
5.2 Офлайн-режим	21
5.3 Обработка ошибок и состояния UI	21
6 ТЕСТИРОВАНИЕ И ДЕМОНСТРАЦИЯ РЕЗУЛЬТАТОВ	22
6.1 Проверенные сценарии	22
6.2 Демонстрационные материалы	22

ВВЕДЕНИЕ

Мессенджеры стали одним из ключевых инструментов цифровой коммуникации: они используются для личного общения, рабочих задач и информационного обмена. От приложений данного класса ожидаются высокая отзывчивость интерфейса, устойчивость к нестабильному соединению и корректная работа фоновых процессов. Поэтому разработка даже учебного «мессенджера» является удобной площадкой для отработки современных подходов Android-разработки.

Цель курсовой работы — разработать Android-приложение MessengerApp с использованием актуального стека технологий, построить масштабируемую архитектуру на основе MVVM и реализовать полный цикл работы с данными: сеть, локальное хранилище, реактивное обновление UI и фоновую синхронизацию.

Для достижения цели были поставлены следующие задачи:

1. Спроектировать пользовательский интерфейс и навигацию между экранами на основе Single Activity и Fragment.
2. Внедрить архитектурный паттерн MVVM и организовать управление состоянием экранов через ViewModel.
3. Реализовать загрузку данных из REST API и их сохранение в локальную базу данных Room для офлайн-доступа.
4. Добавить фоновые задачи синхронизации с использованием WorkManager и систему уведомлений о новых сообщениях.
5. Обеспечить поддержку светлой и темной темы, а также обработку основных состояний UI (loading/empty/error/success).

Объект исследования — платформа Android и ее экосистема библиотек. Предмет исследования — архитектурные паттерны, инструменты Android Jetpack и практики построения устойчивых приложений.

1 ОБЗОР ТЕХНОЛОГИЙ И ИНСТРУМЕНТОВ

1.1 Платформа Android и среда разработки

Приложение разработано для платформы Android. Минимальная поддерживаемая версия — Android 7.0 (API 24), что обеспечивает достаточную аудиторию устройств и одновременно позволяет использовать современные компоненты Jetpack. В качестве основного языка программирования выбран Kotlin, который является рекомендуемым языком для Android-разработки и предоставляет строгую типизацию, лаконичный синтаксис и встроенную поддержку корутин.

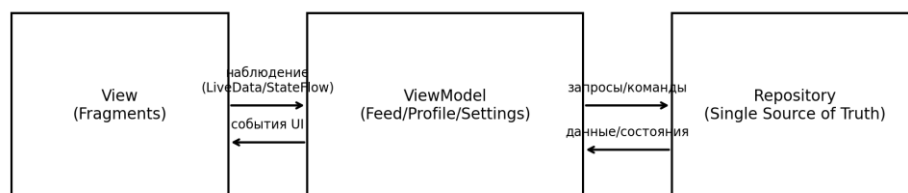
Разработка велась в Android Studio с использованием системы сборки Gradle. Градиентная конфигурация проекта обеспечивает подключение зависимостей, настройки вариантов сборки (debug/release) и параметров компиляции.

1.2 Архитектурный паттерн MVVM

В проекте используется паттерн MVVM (Model-View-ViewModel), который разделяет ответственность между слоями: View отображает состояние и передает пользовательские события, ViewModel хранит UI-состояние и содержит логику экрана, а Model отвечает за получение и хранение данных. Такое разделение упрощает тестирование, снижает связанность компонентов и повышает устойчивость приложения при изменениях интерфейса и источников данных.

Схема взаимодействия компонентов MVVM в MessengerApp показана на рисунке 1.

MVVM в приложении MessengerApp



1.3 Android Jetpack

Android Jetpack представляет набор библиотек, ускоряющих разработку и обеспечивающих совместимость между версиями Android. В рамках проекта использованы следующие ключевые компоненты:

- ViewModel и LiveData - хранение состояния экранов и реактивное обновление UI;
- Navigation Component - декларативная навигация между фрагментами и интеграция с BottomNavigationView;
- Room - ORM-слой над SQLite для локального хранения данных и обеспечения офлайн-режима;
- WorkManager - выполнение отложенных и периодических фоновых задач с учетом ограничений устройства.

1.4 Retrofit и OkHttp

Для сетевого взаимодействия применен стек Retrofit + OkHttp. Retrofit предоставляет декларативное описание REST API и преобразование ответов в модели данных, а OkHttp выполняет HTTP-запросы и поддерживает интерсепторы (например, логирование). Асинхронное выполнение запросов организовано через Kotlin Coroutines.

1.5 Material Design 3

Пользовательский интерфейс построен на основе Material Design 3 (Material You). Использование MaterialCardView, FloatingActionButton, BottomNavigationView и MaterialSwitch обеспечивает единый стиль, адаптивность и корректные анимации взаимодействия. Для визуального оформления применена тема DayNight, позволяющая переключаться между светлым и темным режимами.

1.6 Работа с изображениями

Для отображения аватаров пользователей используются стандартные компоненты ImageView с локальными ресурсами. Архитектура приложения

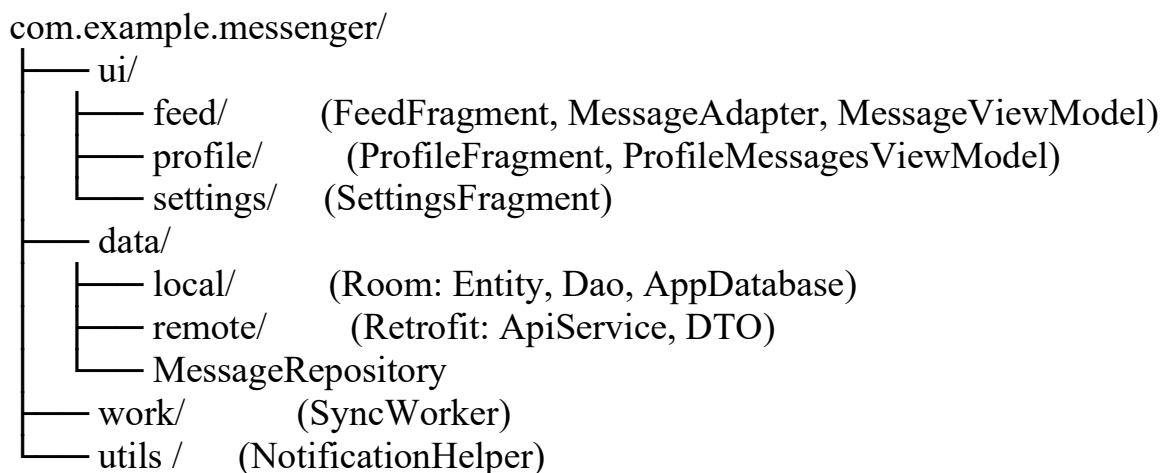
допускает последующее расширение с использованием библиотек загрузки изображений (например, Glide) без изменения основной логики приложения.

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

2.1 Общая структура и пакеты

Проект реализован с разделением на логические слои в рамках единого Android-модуля. Код организован по пакетам в соответствии с архитектурой MVVM, где зависимости направлены от слоя пользовательского интерфейса к слою данных. Доступ к источникам данных осуществляется через репозиторий, выступающий в роли единой точки входа (Single Source of Truth).

Древовидная структура основных пакетов приведена ниже:



2.2 Presentation Layer

Слой представления построен по принципу Single Activity Architecture: приложение содержит одну активность (MainActivity), которая выступает контейнером для фрагментов и отвечает за настройку навигации. Экранная логика вынесена во фрагменты: FeedFragment (лента сообщений), ProfileFragment (профиль) и SettingsFragment (настройки).

2.3 Domain Layer

Слой доменной логики представлен набором ViewModel, каждая из которых отвечает за состояние конкретного экрана.

FeedViewModel управляет загрузкой сообщений, обновлением списка и обработкой пользовательских действий (лайки).

AppViewModel хранит данные профиля пользователя (имя и статус) и используется между фрагментами.

ProfileViewModel отвечает за загрузку и отображение избранных сообщений, сохраненных в локальной базе данных.

2.4 Data Layer

Слой данных объединяет два источника: удаленный (REST API) и локальный (Room). Репозиторий MessageRepository инкапсулирует выбор источника данных, обработку ошибок и кеширование. При наличии сети выполняется загрузка через Retrofit, после чего данные сохраняются в Room. При отсутствии сети используется локальная база данных, что обеспечивает офлайн-доступ.

2.5 Background Layer

Для демонстрации принципов фоновой обработки задач используется WorkManager. Реализация предназначена для иллюстрации периодической синхронизации данных и может быть расширена в дальнейшем.

2.6 Диаграмма потока данных

Схема потока данных при обновлении ленты сообщений представлена на рисунке 2.

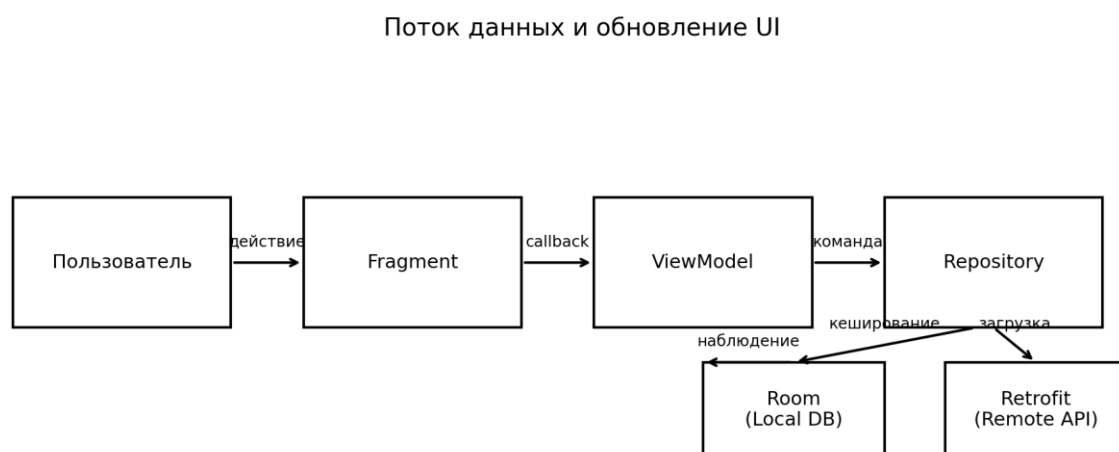


Рисунок 2 - Поток данных и обновление UI

3 РЕАЛИЗАЦИЯ ЛАБОРАТОРНЫХ РАБОТ

3.1 Лабораторная работа №1: базовый UI и Material Design

Цель первого этапа заключалась в создании базового пользовательского интерфейса и навигации, а также в реализации основных элементов взаимодействия (лента сообщений, лайки, переключение темы).

Ключевые результаты этапа:

- создана MainActivity с BottomNavigationView и NavHostFragment;
- реализованы фрагменты FeedFragment, ProfileFragment и SettingsFragment;
- добавлен RecyclerView со списком карточек сообщений на базе MaterialCardView;
- реализована обработка пользовательских действий (клик по элементам ленты);
- подключена тема DayNight и выполнена базовая поддержка светлого и темного режимов.

Конфигурация нижнего меню навигации задается ресурсом bottom_nav_menu.xml (листинг 1).

Листинг 1 - bottom_nav_menu.xml

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/feedFragment"
        android:icon="@android:drawable/ic_menu_agenda"
        android:title="@string/feed" />
    <item
        android:id="@+id/profileFragment"
        android:icon="@android:drawable/ic_menu_myplaces"
        android:title="@string/profile" />
    <item
        android:id="@+id/settingsFragment"
        android:icon="@android:drawable/ic_menu_preferences"
```

```
android:title="@string/settings" />
</menu>
```

Функция «лайка» на данном этапе реализована как обработка пользовательского события на уровне интерфейса. Окончательная логика сохранения состояния была перенесена на следующий этап при интеграции с локальным хранилищем.

3.2 Лабораторная работа №2: интеграция с API и Room

В рамках второго этапа реализована работа с удаленным API и локальным хранилищем. В качестве источника сообщений использован открытый тестовый сервис JSONPlaceholder, предоставляющий данные постов и пользователей в формате JSON.

Для сетевого слоя описан интерфейс ApiService (Retrofit) с методами получения списка постов и пользователей. Полученные данные нормализуются и сохраняются в Room через DAO. Таким образом достигается офлайн-режим: если сеть недоступна, лента заполняется данными из локальной базы.

Пример описания эндпоинта в Retrofit приведен в листинге 2.

Листинг 2 - Интерфейс ApiService

```
interface ApiService {

    @GET("posts")
    suspend fun getMessages(): List<MessageDto>

}
```

Локальное хранилище реализовано с использованием библиотеки Room и включает сущность сообщения (Entity), DAO-интерфейс и класс базы данных. Для упрощения разработки и демонстрации работы с локальным хранилищем на учебном этапе используется стратегия

fallbackToDestructiveMigration, обеспечивающая пересоздание базы данных при изменении схемы.

3.3 Лабораторная работа №3: закрепление API и Room

Третий этап был посвящен закреплению навыков, полученных во второй работе, и расширению уже реализованной функциональности. В частности, было уточнено преобразование DTO в сущности базы данных, обработка ошибок сети и поддержка обновления списка по действию пользователя (кнопка обновления).

3.4 Лабораторная работа №4: WorkManager и уведомления

В рамках лабораторной работы был реализован пример использования WorkManager для периодической фоновой синхронизации данных. Реализация носит демонстрационный характер и иллюстрирует принципы работы с фоновыми задачами и уведомлениями.

Для выполнения фоновой синхронизации используется класс SyncWorker, унаследованный от CoroutineWorker, в котором переопределён метод doWork() (листинг 4).

Листинг 4 - Класс SyncWorker

```
class SyncWorker(  
    context: Context,  
    params: WorkerParameters  
) : CoroutineWorker(context, params) {  
  
    override suspend fun doWork(): Result {  
        val dao = AppDatabase.getDatabase(applicationContext).messageDao()  
        val repository = MessageRepository(dao)  
  
        return try {  
            repository.getMessages()  
            NotificationHelper.showSyncNotification(applicationContext)  
            Result.success()  
        } catch (e: Exception) {  
            Result.failure()  
        }  
    }  
}
```

```

    } catch (e: Exception) {
        Result.retry()
    }
}
}

```

Формирование и отображение уведомления вынесено в отдельный вспомогательный класс NotificationHelper, который инкапсулирует работу с каналами уведомлений и проверку разрешений (листинг 5).

Листинг 5 - Класс NotificationHelper

```

object NotificationHelper {

    private const val CHANNEL_ID = "sync_channel"

    fun showSyncNotification(context: Context) {
        val manager =
            context.getSystemService(Context.NOTIFICATION_SERVICE) as
            NotificationManager

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                CHANNEL_ID,
                "Синхронизация сообщений",
                NotificationManager.IMPORTANCE_DEFAULT
            )
            manager.createNotificationChannel(channel)
        }

        val notification = NotificationCompat.Builder(context, CHANNEL_ID)
            .setSmallIcon(R.mipmap.ic_launcher)

```

```

        .setTitle("Сообщения")
        .setText("Новые данные получены")
        .setAutoCancel(true)
        .build()

        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU ||

context.checkSelfPermission(android.Manifest.permission.POST_NOTIFICATION
S)
        == PackageManager.PERMISSION_GRANTED
    ) {
        manager.notify(1, notification)
    }
}
}

```

4 ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

4.1 Главный экран - лента сообщений

Главный экран приложения представляет ленту сообщений, реализованную с помощью RecyclerView. Каждый элемент списка отображается в виде карточки (MaterialCardView) и содержит аватар пользователя, имя, e-mail, заголовок и краткий текст сообщения. Длина текста ограничивается, чтобы элементы списка оставались компактными. В правой части карточки размещена кнопка «лайк» в виде иконки сердца.

Для инициирования обновления списка предусмотрена FloatingActionButton с иконкой обновления. Во время загрузки показывается индикатор (ProgressBar), а при отсутствии данных - пустое состояние (Empty state).

Примеры отображения ленты представлены на рисунках 3 и 4.

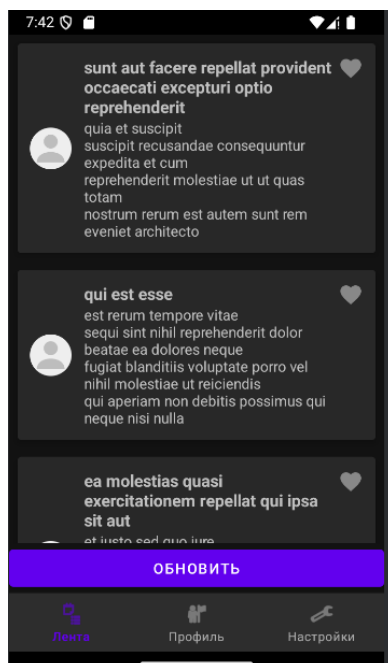


Рисунок 3 - Лента сообщений в темной теме

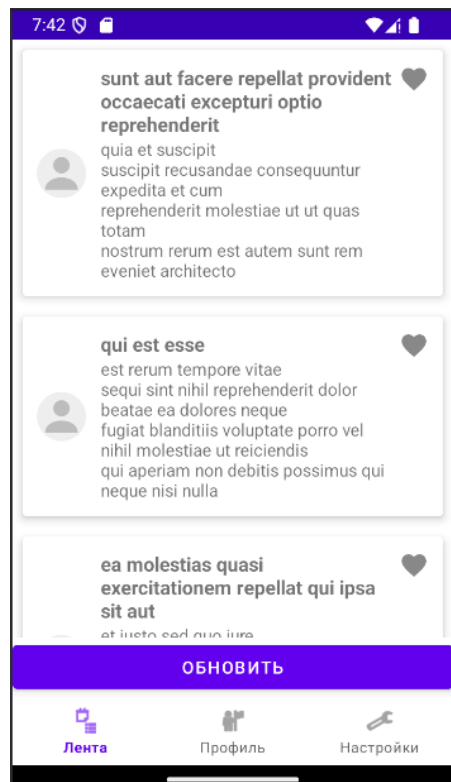


Рисунок 4 - Лента сообщений в светлой теме

4.2 Экран профиля

Экран профиля содержит два поля ввода: имя и статус пользователя. Данные профиля хранятся в `ProfileViewModel`, что позволяет сохранять состояние при повороте экрана и пересоздании фрагмента. Обновление значений может выполняться по событию потери фокуса или по нажатию кнопки сохранения (в зависимости от выбранного сценария). Ниже полей профиля отображается список избранных сообщений, реализованный с помощью `RecyclerView`. В данный список попадают сообщения, отмеченные пользователем как «понравившиеся» в ленте. Каждая карточка в профиле также содержит иконку лайка, позволяющую удалить сообщение из избранного. При снятии лайка элемент автоматически исчезает из списка, так как данные обновляются через локальную базу `Room` и реактивно передаются в UI.

Пример экрана профиля приведен на рисунке 5.

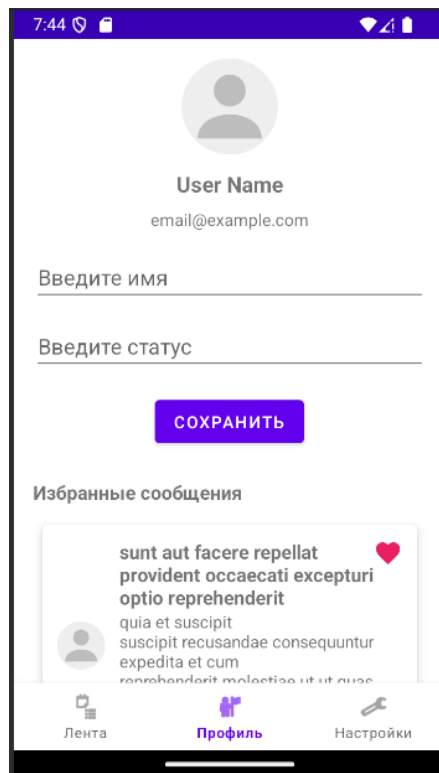


Рисунок 5 - Экран профиля

4.3 Экран настроек

Экран настроек содержит переключатель темной темы (MaterialSwitch). Состояние темы хранится в SettingsViewModel и при изменении немедленно применяется ко всему приложению через AppCompatActivity.setDefaultNightMode(). При необходимости выбор темы сохраняется в SharedPreferences, что обеспечивает восстановление режима после перезапуска приложения.

Пример экрана настроек приведен на рисунке 6.



Рисунок 6 - Экран настроек (переключатель темной темы)

4.4 Bottom Navigation

Навигация между экранами реализована через BottomNavigationView и Navigation Component. Панель содержит три вкладки: «Лента», «Профиль» и «Настройки». Переходы выполняются через NavController, а навигационный граф (nav_graph.xml) описывает доступные направления.

Наличие нижней навигации обеспечивает быстрый доступ к основным разделам приложения и соответствует рекомендациям Material Design.

4.5 Уведомления

После успешной фоновой синхронизации приложение уведомляет пользователя о получении новых данных. Уведомление носит информационный характер и подтверждает успешное обновление локального хранилища. Пример системного уведомления приведен на рисунке 7. Пример запроса разрешения на отправку уведомлений приведен на рисунке 8.

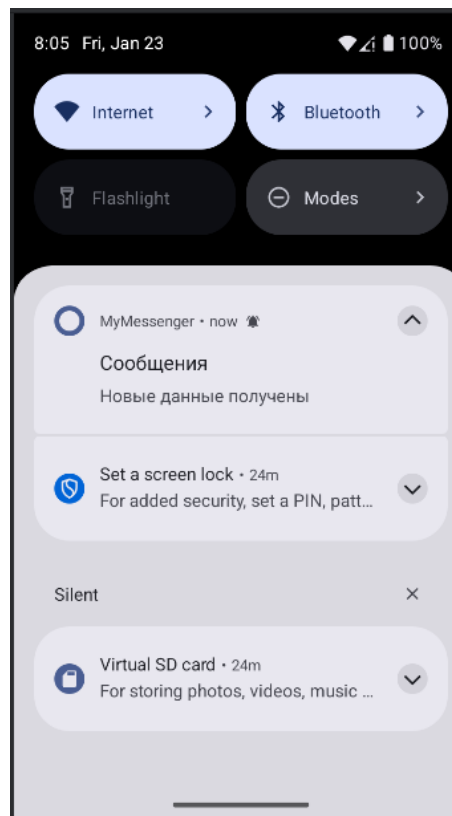


Рисунок 7 - Системное уведомление о получении новых сообщений

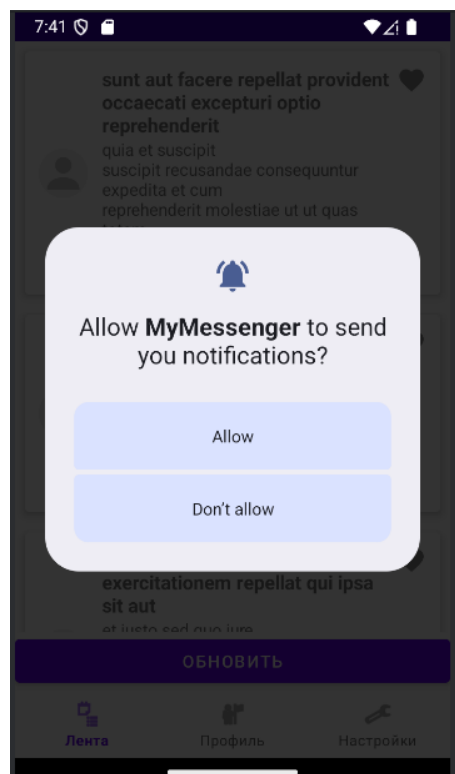


Рисунок 8 – Запрос разрешения на отправку уведомлений

5 ОСОБЕННОСТИ РЕАЛИЗАЦИИ

5.1 Кеширование и Single Source of Truth

В приложении реализовано кеширование данных на уровне локального хранилища Room. Все сообщения, полученные из сети, сохраняются в базе данных и используются как основной источник данных для пользовательского интерфейса. Такой подход соответствует принципу Single Source of Truth, при котором UI работает с согласованным источником данных независимо от их происхождения (сеть или локальное хранилище).

Подход Single Source of Truth означает, что UI получает данные из одного согласованного источника - локальной базы/репозитория. Даже если данные приходят из сети, они сначала нормализуются и сохраняются в базу, а затем наблюдаемые объекты UI получают обновления реактивно.

5.2 Офлайн-режим

Офлайн-режим обеспечивается за счет хранения данных в Room и корректного выбора источника данных в репозитории. При отсутствии подключения приложение продолжает отображать последний успешный снимок данных. Обновление списка при отсутствии сети не приводит к падению приложения и может сопровождаться уведомлением пользователя об ошибке загрузки.

5.3 Обработка ошибок и состояния UI

Для повышения устойчивости приложения введены состояния загрузки: loading, success, empty и error. ViewModel управляет данными экрана и процессом загрузки, а UI отображает индикатор загрузки, список сообщений или сообщение об отсутствии данных в зависимости от текущего состояния.

6 ТЕСТИРОВАНИЕ И ДЕМОНСТРАЦИЯ РЕЗУЛЬТАТОВ

6.1 Проверенные сценарии

В ходе разработки были проверены следующие пользовательские сценарии:

1. Запуск приложения и переход между вкладками нижней навигации.
2. Загрузка сообщений при наличии сети и корректное отображение списка.
3. Обновление списка по нажатию `FloatingActionButton`.
4. Сохранение и восстановление данных профиля при повороте экрана.
5. Переключение светлой/темной темы и сохранение выбора пользователя.
6. Работа в офлайн-режиме: запуск приложения без сети и отображение данных из `Room`.
7. Периодическая фоновая синхронизация данных и отображение системного уведомления об успешном обновлении.

6.2 Демонстрационные материалы

Скриншоты основных экранов и системного уведомления приведены в Приложении А. Фрагменты ключевого кода (репозиторий, миграции, адаптер, воркер синхронизации и `ViewModel`) приведены в Приложении Б.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы разработано Android-приложение MessengerApp, демонстрирующее комплексный подход к мобильной разработке: пользовательский интерфейс на Material Design 3, архитектуру MVVM, демонстрирующее использование сетевого слоя и локального хранения данных, локальное хранение в Room, фоновые задачи WorkManager и уведомления пользователя.

Построенная архитектура обеспечивает масштабируемость и удобство поддержки: логику работы с данными можно расширять (например, добавлять отправку сообщений, авторизацию или пагинацию) без существенной переработки UI. Реализованные механизмы кеширования и офлайн-режима повышают надежность приложения и соответствуют ожиданиям к программам класса «мессенджер».

В качестве возможных направлений развития можно выделить:

- добавление авторизации и реальных диалогов (чаты, отправка сообщений);
- поиск, фильтрация и пагинация для больших списков;
- пуш-уведомления через Firebase Cloud Messaging;
- покрытие проекта unit- и UI-тестами, настройка CI/CD;
- переход на Jetpack Compose или использование Kotlin Multiplatform для iOS-версии.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Android Developers. Documentation. - URL: <https://developer.android.com/> (дата обращения: 21.01.2026).
2. Android Developers. Guide to app architecture. - URL: <https://developer.android.com/topic/architecture> (дата обращения: 21.01.2026).
3. Android Developers. Room Persistence Library. - URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 21.01.2026).
4. Android Developers. WorkManager. - URL: <https://developer.android.com/topic/libraries/architecture/workmanager> (дата обращения: 21.01.2026).
5. Android Developers. Navigation component. - URL: <https://developer.android.com/guide/navigation> (дата обращения: 21.01.2026).
6. Kotlin Documentation. - URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 21.01.2026).
7. Square. Retrofit. - URL: <https://square.github.io/retrofit/> (дата обращения: 21.01.2026).
8. Square. OkHttp. - URL: <https://square.github.io/okhttp/> (дата обращения: 21.01.2026).
9. Bumptech. Glide. - URL: <https://bumptech.github.io/glide/> (дата обращения: 21.01.2026).
10. Material Design 3. - URL: <https://m3.material.io/> (дата обращения: 21.01.2026).
11. JSONPlaceholder. Free fake API for testing and prototyping. - URL: <https://jsonplaceholder.typicode.com/> (дата обращения: 21.01.2026).